QR CODE THREAT SIGNATURES & TRAINING GUIDE

A practical, defensive reference for building a QR scanner that explains risk, checks destinations, and avoids false confidence

Scope: QR payload analysis • URL and phishing indicators • mobile action risks • safe dataset design • detection workflow • validation and evaluation

Important: A QR image does not usually contain a conventional malware signature. Most QR-related harm happens after scanning—when a person opens a deceptive website, installs an app, approves a payment, joins a network, or follows a device action. Treat the QR payload and destination as evidence to assess, not proof by themselves.

Prepared: 28 October 2026 • Defensive / educational use

1. How to use this guide

Use this document as a taxonomy and feature checklist for your QR scanner—not as a ready-made database of every malicious QR code. Threats change constantly, and there is no finite universal list of QR malware signatures. A reliable detector combines local rules, reputation services, safe parsing, and clear user warnings.

- **Decode first:** extract the QR payload as text without automatically opening it or executing an action.
- **Classify the payload:** URL, app/deep link, Wi-Fi configuration, contact card, email/SMS/phone action, payment URI, plain text, or unknown format.
- **Inspect and score:** check structural red flags, destination reputation, redirects (using a safe server-side scanner), and context.
- **Explain the evidence:** show the exact reason for a warning; do not simply display an unexplained red/green label.
- **Fail safely:** if the destination cannot be checked, say “Unknown / not verified,” not “Safe.”

Terminology: a **signature** can mean a known bad URL/hash from a threat feed, a recognizable pattern, or a behavioral indicator. QR security needs all three types of evidence.

2. Threat taxonomy: what QR codes can lead to

Threat / payload category	What it means	Useful detection signals (not proof)
Phishing / quishing	A QR code sends the user to a fake sign-in, delivery, bank, payment, school, or workplace page.	Known phishing reputation; brand/domain mismatch; lookalike spelling; unexpected credential request; misleading page title or URL path.
Malware delivery	Destination offers a malicious app, file, profile, or other download.	Known malicious URL/file hash; unexpected APK or executable download; newly seen or low-reputation host; deceptive download button.
Unwanted software / PUA	Installer or site pushes intrusive, deceptive, or unwanted software.	Threat-feed match; deceptive installer claims; bundled downloads; excessive permissions described by trusted analysis.
Credential / session theft	Fake page asks for passwords, one-time codes, recovery codes, or session details.	Login form on unrelated domain; urgent account-warning language; request for OTP or recovery information; brand mismatch.
Payment / QR replacement fraud	A substituted QR changes the payee, amount, or payment destination.	Payment URI fields differ from trusted expected values; recipient mismatch; unusual amount; code pasted over a legitimate one.
Redirect chain / cloaking	A short link or intermediate page forwards to another destination.	Shortener; multiple redirects; cross-domain hops; destination differs from displayed/expected brand; final destination unavailable.
Lookalike / IDN homograph	A domain visually resembles a trusted domain, including Unicode lookalikes.	Punycode/IDN; mixed scripts; edit-distance similarity to a protected brand; unexpected top-level domain.
Deep-link / app action abuse	A URI launches an app or passes data into an app action.	Non-HTTP scheme; app/package identifier mismatch; sensitive action parameters; unknown custom scheme; action would happen without a clear confirmation.
Wi-Fi configuration abuse	QR offers to join a wireless network.	Unexpected network name; open or weakly protected network; network name impersonates a trusted venue; scanner attempts to auto-join.
Contact / calendar / message abuse	QR imports a contact/event or prepares an email, SMS, or phone call.	Unexpected links or instructions in fields; suspicious phone/email; prefilled message requests money or secrets; calendar event contains deceptive URLs.
Fake support / tech-support scam	Destination claims the device is infected or asks the user to call support.	Alarmist warning; fake scan results; phone number unrelated to official support; request to install remote-access software.
Tracking / privacy abuse	QR redirects through tracking links or collects data.	Unnecessary tracking parameters; multiple third-party redirects; request for location, contacts, camera, or other data unrelated to purpose.
Exploit / drive-by attempt	A vulnerable browser or app may be targeted after opening a page.	Threat intelligence, sandbox/browser telemetry, exploit indicators from specialist tools; a QR-only static scan cannot reliably prove exploit behavior.

Threat / payload category	What it means	Useful detection signals (not proof)
QR tampering / substitution	An attacker places a sticker or replacement code over a legitimate one.	Visual overlay, damaged packaging, mismatch with printed instructions, unexpected payment recipient; requires physical/context checks.
Scanner-app abuse	A malicious scanner app—not necessarily the QR itself—misuses permissions or displays deceptive results.	Excessive permissions, unknown publisher, poor provenance, suspicious app reputation; app-store and device security checks.
Plain-text social engineering	The QR contains instructions or a URL disguised as harmless text.	Urgent requests, payment/OTP requests, encoded or obscured text, commands that ask user to disable protections.

3. URL and payload indicators to extract

These are features for a risk model. Most are **weak signals alone**; combine several, check threat intelligence, and avoid treating a single feature as proof.

Feature group	Examples to detect	Recommended handling
Scheme / protocol	http, https, ftp, file, intent, javascript-like text, custom app schemes, payment schemes, mailto, tel, sms, Wi-Fi payloads.	Allow-list supported schemes. Never execute or launch a decoded action automatically. Treat unsupported schemes as review-required.
Domain structure	IP-literal host, unusually long hostname, many subdomains, punycode, mixed scripts, misspelled brand, odd TLD.	Normalize safely; display the registrable domain prominently; compare against a maintained brand allow-list; avoid simplistic TLD blocking.
URL obfuscation	Percent encoding, nested encoding, @ in authority, backslashes, unusual ports, excessive query length, embedded URL parameters.	Parse with a standards-compliant URL parser; canonicalize carefully; flag ambiguous or malformed forms; do not use string matching alone.
Shorteners / redirects	Known URL shortener, redirect parameters, several domains in chain, final URL different from expected domain.	Resolve only in a sandboxed backend with strict time, hop, DNS, and private-IP protections; do not let the scanner visit internal network addresses.
Path / query semantics	login, verify, wallet, claim, reset, download, apk, update, invoice, urgent; token-like query parameters.	Use as low-confidence contextual features; words can occur on legitimate pages. Never log secrets or full sensitive query strings unnecessarily.
Download indicators	APK, EXE, MSI, DMG, ZIP, profile/config download, unexpected MIME type or file extension.	Warn before download; do not download or install automatically; use reputable file reputation scanning if a file is explicitly submitted.
Reputation / age	Threat-feed hit, recently registered domain (if a trusted data source supplies it), newly observed URL, hosting abuse reports.	Use reputable, current feeds; keep source and timestamp; treat absence from a feed as “not known bad,” not as safe.
Brand and context	QR says “official bank” but domain is unrelated; payment code on a replaced poster; destination conflicts with venue or stated purpose.	Show domain/payee/amount clearly and ask the user to verify through a trusted channel.
Payload size / encoding	Very long payload, nested Base64-like text, control characters, invisible Unicode, unusual binary/custom format.	Bound payload length; sanitize display; preserve original for analysis; decode only known safe formats and never evaluate decoded text as code.
QR image manipulation	Logo overlay, sticker edges, poor print, code covers another code, altered destination in a payment poster.	Image heuristics can flag possible tampering but are not definitive. Add a user checklist for physical inspection.

4. Known threat intelligence and signature sources

Use these sources for current URL reputation and official threat categories. Check licensing, quotas, privacy terms, and product status before integrating. APIs generally classify URLs or files—not the QR image itself.

Source	Useful role	Integration notes
Google Safe Browsing / Web Risk	Detects known unsafe web resources, including malware, social engineering/phishing, and unwanted software.	Use the currently supported API and threat types. Do not expose API secrets in a public website or mobile client. Read privacy and billing terms.
VirusTotal	Aggregates security-vendor verdicts for submitted URLs/files and provides reports.	Useful as an additional signal, not a sole decision maker. Public API quotas and permitted use may be restrictive; do not submit private URLs/files without considering privacy.
Official browser / OS protections	Browser warnings, app-install protection, operating-system security checks.	Use alongside your own analysis. Do not attempt to bypass platform warnings.
CERT / national cyber security advisories	Campaign information and advisories relevant to a region or sector.	Use advisories as context and update detection rules carefully.
Your own reviewed dataset	Labeled benign, phishing, malware-delivery, suspicious, and unknown samples with provenance.	Store source, collection date, label confidence, and review status. Never treat synthetic examples as real threat-feed matches.

References: Google Safe Browsing threat types and overview; Google Cloud Web Risk threat lists/evaluate API; VirusTotal API overview; Government of Canada, “Security considerations for QR codes”; European Commission QR Code Security Guide.

5. Suggested training labels

Label	Definition	User-facing result
BENIGN_VERIFIED	Expected benign payload and destination checked with trusted evidence.	No known issue detected; still verify payee or action.
PHISHING	Trusted source confirms credential-harvesting or deceptive page.	High risk — likely phishing; do not enter credentials.
MALWARE_DELIVERY	Trusted source confirms malware distribution or malicious file delivery.	High risk — known malware-related destination.
UNWANTED_SOFTWARE	Trusted source confirms unwanted software distribution.	Warning — unwanted software risk.
PAYMENT_MISMATCH	Payment destination/recipient/amount conflicts with trusted expected details.	Stop and verify recipient and amount before paying.
SUSPICIOUS	Multiple risk indicators but not confirmed malicious.	Caution — suspicious indicators found; destination not confirmed malicious.
UNKNOWN_UNCHECKED	No reliable verdict, offline, unsupported format, or lookup failure.	Unknown — could not verify. Do not call it safe.
UNSUPPORTED_ACTION	Payload would trigger an unsupported or sensitive action.	Blocked from automatic action; user can inspect details.

6. Safe dataset schema for model training

Keep one row per scanned sample or observation. Use a stable sample ID and avoid storing personal or payment data. Store only the minimum needed for analysis.

Field	Purpose / example
sample_id	Random internal identifier; never use a person's identity.
payload_type	https_url, wifi, vcard, sms, payment_uri, plain_text, custom_scheme, unknown.
raw_payload_private	Optional restricted field; encrypt and redact secrets. Prefer a sanitized payload for training.
normalized_host	Canonical hostname only, when applicable.
url_features	Length, scheme, host structure, IDN/punycode flag, IP-host flag, redirect count, encoding indicators.

Field	Purpose / example
reputation_verdict	Provider name, result, checked_at timestamp, threat category, and confidence if supplied.
label	One of the labels above.
label_source	Threat feed, expert review, user report pending review, or synthetic test.
label_confidence	confirmed / high / medium / low; do not mix uncertain reports with confirmed samples.
first_seen / last_checked	Dates needed to manage stale labels.
split_group	Domain/campaign grouping to prevent near-duplicate leakage across train/test splits.
notes	Short evidence summary; no passwords, OTPs, session tokens, full bank data, or unnecessary personal data.

Dataset quality rules

- Separate confirmed malicious, suspicious, benign, and unknown examples. Do not label all unfamiliar URLs as malicious.
- Use domain- or campaign-grouped train/test splits; otherwise near-duplicate URLs can make accuracy look falsely high.
- Deduplicate normalized URLs and keep the original provenance separately.
- Use time-based evaluation as well as random splits, because attacker infrastructure and tactics change.
- Record label age and recheck stale samples. A previously malicious URL can become inactive; a previously benign URL can be compromised.
- Include difficult benign cases: legitimate short links, long tracking URLs, internationalized domains, login pages, payment links, and uncommon but valid schemes.
- Use synthetic samples only to test parsing and UI behavior; mark them synthetic and do not claim they represent live malware.

7. Recommended scanner pipeline

Stage	Action	Safe design requirement
1. Capture	Read QR image and decode payload.	Camera permission only when needed; do not open links automatically.
2. Parse	Identify payload type and parse with a safe library.	Bound size; handle malformed data; no eval, shell execution, or automatic intent launch.
3. Normalize	Canonicalize host and URL for analysis while preserving original for display.	Use standards-compliant parsing; defend against Unicode and encoding edge cases.
4. Local heuristics	Calculate scheme, host, obfuscation, brand mismatch, action and context features.	Rules should produce reasons and weights, not absolute claims.
5. Reputation lookup	Query trusted URL threat-intelligence services from a protected backend.	Protect keys; set timeouts; handle quota/network failures; minimize data sent.
6. Safe redirect analysis	If needed, resolve redirects in isolated infrastructure.	Block localhost, private IP ranges, link-local, metadata endpoints, and DNS rebinding; cap hops and time.
7. Risk decision	Combine threat-feed verdicts, features, and confidence.	Known bad > suspicious indicators > unknown. Do not equate “no hit” with “safe.”
8. Explain	Show decoded action, destination domain, evidence, and next safe step.	Avoid exposing secrets; provide a clear reason and safe exit.
9. Feedback	Allow report/correction, but treat user reports as unverified until reviewed.	Rate-limit reports; prevent poisoning; preserve audit trail.

8. Example scoring policy (starting point, not a universal rule)

Start with explicit hard blocks for confirmed threat-feed matches and unsafe actions, plus a risk score for weaker signals. Calibrate using a labeled validation set before deployment.

Evidence	Illustrative score	Interpretation
Confirmed threat-feed match	Hard high-risk verdict	Strongest available evidence; show provider and check time.
Payment recipient/amount mismatch	Hard stop / user confirmation required	Do not attempt to infer that a payment is safe from URL reputation alone.
Untrusted custom scheme or automatic sensitive action	Block automatic launch	Show payload details safely.
Multiple independent domain/URL red flags	Add moderate risk points	Suspicious, not necessarily malicious.
One weak indicator (e.g., URL shortener)	Small risk points	Many legitimate services use short links.
No reputation hit and no heuristics	No positive risk evidence	Return “No known threat detected,” not “Guaranteed safe.”

Do not copy arbitrary numeric thresholds from this guide into production. Choose thresholds from measured false-positive/false-negative trade-offs and the cost of each error.

9. What a QR scanner can and cannot tell

- **Can often tell:** what data is encoded; whether it is a URL or an action; obvious malformed payloads; suspicious URL structure; known bad reputation; whether the destination differs from the expected domain or payee when you have a trusted reference.
- **Cannot reliably tell from the QR pixels alone:** every future malware family; whether a website will later change; whether a clean-looking site will steal data after login; whether a device is already infected; whether a payment recipient is trustworthy without context.
- **Static scan limitation:** a URL may be benign at scan time and become malicious later, or may serve different content by region, device, time, or redirect path.
- **Image-model limitation:** image appearance, logo similarity, and print defects can support tamper suspicion but cannot prove that the destination is safe or malicious.
- **Best wording:** “Known malicious destination,” “Suspicious indicators,” “Unknown / not verified,” or “No known threat detected.” Avoid absolute guarantees.

10. Test plan and evaluation

Test group	What to test	Expected behavior
Basic payloads	Plain text, HTTPS URL, Wi-Fi, vCard, calendar, mailto, tel, SMS, payment URI.	Correct type detection; no action runs automatically.
Malformed input	Empty payload, very long payload, invalid encoding, control characters, unsupported format.	Graceful error, bounded processing, safe display.
URL normalization	Upper/lowercase host, percent encoding, Unicode domain, punycode, user-info syntax, unusual port.	Consistent parsing; ambiguous URLs flagged; no parser crash.
Threat reputation	Known test indicators supplied by a trusted provider; offline lookup.	Correct verdict and source/time; lookup failure becomes unknown.
Redirect safety	Shorteners, redirect loops, excessive hops, private IP target, DNS changes.	Hop/time limits; internal destinations blocked; no SSRF exposure.
Payments	Known test payment QR and deliberately mismatched mock recipient/amount.	Show recipient and amount; stop on mismatch when trusted expected data exists.

Test group	What to test	Expected behavior
Adversarial benign	Legitimate long URLs, short links, non-English domains, valid uncommon schemes.	Measure false positives and explain uncertainty.
Privacy/security	API keys, logs, URL query secrets, analytics, permissions.	Keys stay server-side; sensitive fields redacted; minimum permissions.
Model performance	Precision, recall, false-positive rate, false-negative rate, calibration, latency.	Report metrics by class and by payload type, not only overall accuracy.

Recommended metrics: precision = true positives / predicted positives; recall = true positives / all actual positives; false-positive rate = benign samples incorrectly flagged / all benign samples. For a safety scanner, report both missed threats and false alarms, plus the number of unknown samples.

11. Privacy, security, and product design checklist

- Never send a scanned URL to a third party without disclosing it and considering that URLs can contain private tokens or identifiers.
- Keep API credentials on a server-side component, not in public JavaScript, an APK, or a downloadable app bundle.
- Do not automatically open URLs, install files, join Wi-Fi, call numbers, send messages, import contacts, or initiate payments.
- Use HTTPS for your own app; authenticate administrative functions; rate-limit lookups; validate all server inputs.
- If you implement redirect resolution, isolate it from internal services and deny private/link-local destinations to prevent server-side request forgery.
- Use minimal camera/storage/network permissions and explain why each is required.
- Display the full registrable domain and the exact action; for payment codes show payee and amount before the user leaves the scanner.
- Keep audit logs free of credentials, OTPs, session cookies, private message contents, and unnecessary full query strings.
- Add a visible “could not verify” state for offline, timeout, unsupported format, or API quota errors.
- Do not claim that your model is trained on “all malware signatures.” State the sources, update schedule, measured performance, and limitations.

12. Implementation checklist

- Decode library: reliable QR decoder; payload parsing isolated from action handling.
- Backend: URL normalization, reputation adapter, timeouts, rate limiting, key management, privacy-safe logging.
- Detection engine: explicit known-bad matches plus interpretable heuristics and a calibrated model if enough labeled data exists.
- Threat feeds: source, licence, quota, freshness, and provider outage behavior documented.
- UI: risk level, decoded type, destination, evidence, timestamp, and safe next step.
- Dataset: provenance, confidence, time-aware evaluation, deduplication, grouped split, and review workflow.
- Release: security review, abuse testing, privacy notice, false-positive monitoring, and incident response plan.

13. Sources and further reading

Google Safe Browsing API — overview
https://developers.google.com/safe-browsing/v4

Google Safe Browsing — threat types
https://developers.google.com/safe-browsing/v4/reference/rest/v4/ThreatType

Google Cloud Web Risk — threat lists
https://docs.cloud.google.com/web-risk/docs/lists

Google Cloud Web Risk — Evaluate API
https://docs.cloud.google.com/web-risk/docs/evaluate-api

VirusTotal API overview
https://docs.virustotal.com/docs/api-overview

Government of Canada — Security considerations for QR codes (PDF)
https://www.cyber.gc.ca/sites/default/files/itsap00141-security-considerations-qr-codes-v2.pdf

European Commission — QR Code Security Guide
https://ec.europa.eu/digital-building-blocks/sites/spaces/DIGITAL/pages/905217084/QR+code+Security+Guide

Source note: threat categories and API capabilities can change. Check the official documentation before building against a particular endpoint. This guide is a defensive engineering reference, not a guarantee of detection.